Java OOP Laboratory Exercise: Campus Cafe Ordering System

Student name: Christoff Allen B. Jumapit

Section: C

Date: November 10, 2026

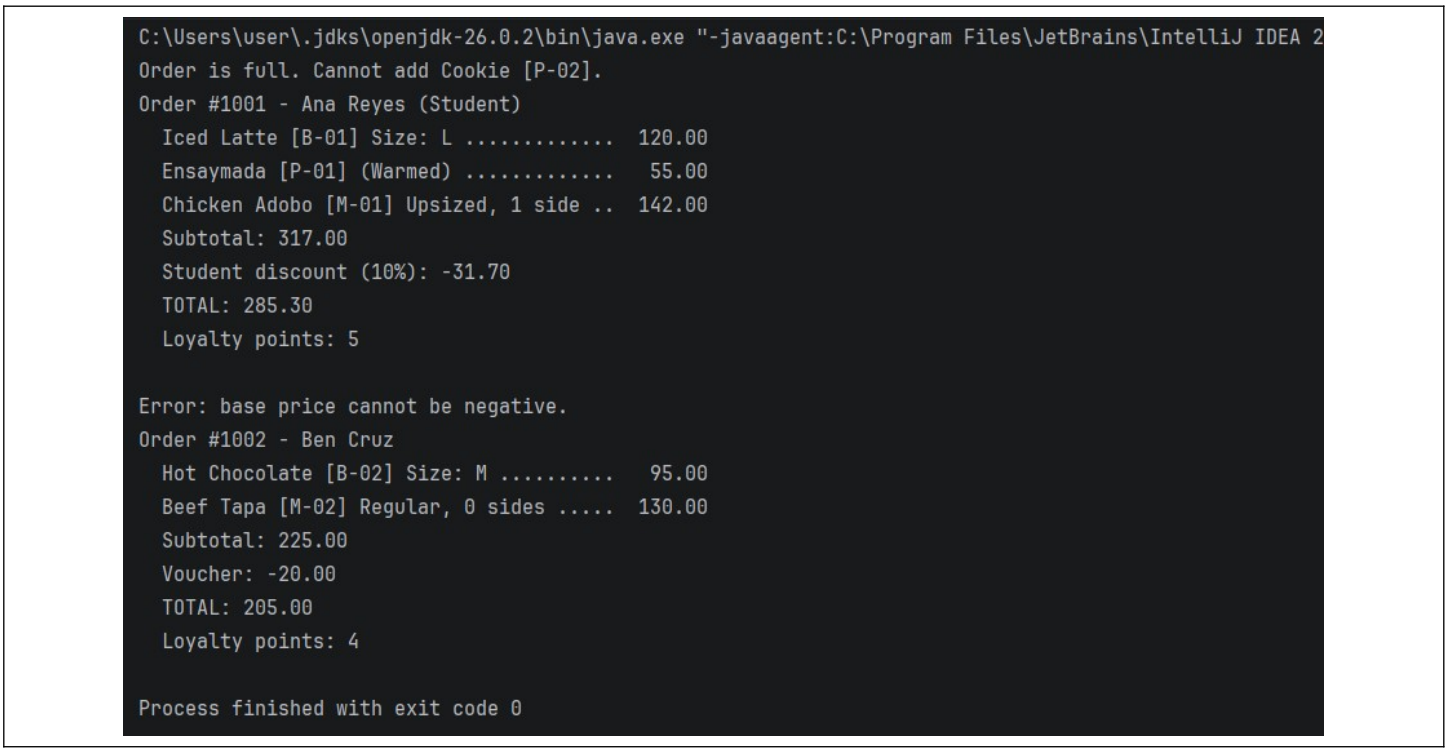
Exercise title: Java OOP Laboratory Exercise: Campus Cafe Ordering System

Concept Explanation

Concept	Where and why it was used
If statement	Applied in Pastry.computePrice() to add the warming fee and in Meal.computePrice() to add the upsize fee. A single if statement keeps each fee conditional on its own flag.
If-else statement	Applied in MenuItem.setBasePrice(), Beverage.computePrice(), Order.addItem(), and Order.computeTotal(). These methods must choose between two or more outcomes, such as accepting or rejecting a value.
Classes and objects	The system is divided into seven classes, each with one clear purpose. CafeApp.main() creates the objects from these classes and runs the program.
Attributes and behaviors	Every class declares attributes that store its data and methods that act on that data. For example, Customer stores loyaltyPoints and provides addPoints().
Object interaction	Order holds one Customer and up to three MenuItem objects. printReceipt() requests each item's price and updates the customer's loyalty points.
Java naming conventions	Class names use PascalCase, methods and variables use camelCase, and constants such as MAX_ITEMS use UPPER_SNAKE_CASE. Consistent names make the code easier to read and maintain.
Default constructor	MenuItem(), Beverage(), Pastry(), Meal(), and Customer() create an object without any arguments. CafeApp uses the item versions to create Hot Chocolate, Cookie, and Beef Tapa before the setters fill in their details.
Parameterized constructor	The three item classes, Customer, and Order accept their initial values as arguments. This allows an object to be created in a valid state in a single statement.
Constructor overloading	Customer provides Customer(), Customer(String), and Customer(String, boolean). Each version supports a different amount of known information about the customer.
public, private, protected	Fields are private, except name and basePrice in MenuItem, which are protected so that subclasses can use them directly. Methods are public so that other classes can call them.
Getters, setters, data hiding	Fields can be read and changed only through getters and setters. The setters validate input, such as rejecting a negative price or an invalid size, so the data stays correct.
Method overloading	Customer.addPoints(int) and addPoints(double) share a name but differ in parameters. Order.computeTotal() and computeTotal(double) follow the same pattern, with the second version applying a voucher.
Method overriding	Beverage, Pastry, and Meal override computePrice() and getDescription() using @Override. Each menu item type therefore calculates its own price and builds its own description.
Abstraction	MenuItem is an abstract class with the abstract method computePrice(). It cannot be instantiated, and each subclass is required to supply its own pricing rule.

Screenshots of the Program Output

The following screenshot shows the actual output of the program when CafeApp is run. It includes both receipts, the rejection of the fourth item, and the negative-price error message.



Challenges Encountered and How I Solved Them

- Storing items without a collection. The assignment prohibits arrays and collections, so the items are kept in three separate fields (item1, item2, and item3). An if-else-if chain in addItem() places each item in the first empty field, and itemCount enforces the limit of three.
- Displaying the voucher. The required attributes of Order do not include a voucher. An overloaded printReceipt(double voucher) method was added, and printReceipt() calls it with a value of 0.
- Validation in constructors. Assigning a value directly in a constructor bypasses the checks of its setter. The constructors therefore set a safe starting value and then call the setter.
- Loyalty points. Points are whole numbers, so the amount spent is divided by 50 and cast to int. A total of 285.30 pesos earns 5 points and 205.00 pesos earns 4 points, which agrees with the sample output.